Ders 5 Genişletilmiş Notu: Büyük Veride Kümeleme: Hiyerarşik, K-Means, BFR ve CURE

Ders 5 Genişletilmiş Notu: Büyük Veride Kümeleme: Hiyerarşik, K-Means, BFR ve CURE

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Kümeleme Nedir?
- 2. Kümeleme Neden Zordur?
- 3. Uygulama Örnekleri
- 4. Uzaklık Ölçüleri
- 5. Kümeleme Yöntemlerinin İki Ana Ailesi
- 6. Hiyerarşik Kümeleme
- 7. Birleştirici Kümeleme Üzerine Adım Adım Örnek
- 8. Ne Zaman Durmalı? Küme Kalitesi
- 9. Öklid Dışı Uzay ve Clustroid
- 10. Hiyerarşik Kümelemenin Karmaşıklığı
- 11. K-Means Algoritması
- 12. k Değerini Seçmek
- 13. Başlangıç Noktalarını Seçmek
- 14. K-Means'in Karmaşıklığı
- 15. BFR Algoritması
- 16. BFR'de Özet İstatistikler: N, SUM, SUMSQ
- 17. Mahalanobis Uzaklığı ve Eşik Seçimi
- 18. CS Kümelerinin Birleştirilmesi ve Algoritmanın Sonu
- 19. CURE Algoritması
- Örnek Sorular
- Dersin Toparlaması

1. Kümeleme Nedir?

Hoca derse, kitabın 7. bölümüne (chapter 7), yani büyük ölçekli veri kümelerinde kümelemeye (clustering) geçtiğimizi söyleyerek başladı. Şimdiye kadar 1, 2 ve 3. bölümler işlenmişti. Ders malzemeleri kitabın web sitesinde ve dersin web sayfasının doküman kısmında bulunuyor.

Kümeleme bir makine öğrenmesi tekniğidir ve gözetimsiz öğrenme (unsupervised learning) olarak da bilinir. Amaç, veritabanlarında bulunan verinin altında yatan yapıyı (underlying data structure) anlamaktır. Yapıdan kastedilen, verinin nasıl biriktiği, verinin içindeki bölümlerin (segmentation) neler olduğudur. Bu bölümleri bulursak elimizdeki veri hakkında bir fikir sahibi oluruz.

Hoca bunu iki boyutlu bir resimle gösterdi. Resimde açıkça üç ayrı yapı vardı. Her yapının içindeki noktalar birbirine yakındı. Bunlardan uzakta kalan iki ayrı nokta da vardı. Kümeleme algoritmasını bu veriye uygularsak üç küme kusursuz biçimde bulunur. Böylece veride üç belirgin bölüm olduğunu söyleyebiliriz: bölüm 1, bölüm 2 ve bölüm 3.

Biçimsel tanım şöyledir: Aralarında bir uzaklık kavramı (notion of distance) tanımlı bir noktalar kümesi verilsin. Bu noktaları belli sayıda kümeye (cluster) gruplamak istiyoruz. Kümeyi şöyle tanımlarız:

- Aynı kümenin üyeleri birbirine yakındır, yani benzerdir.
- Farklı kümelerin üyeleri birbirinden farklıdır, aralarında bir uzaklık vardır.

2. Kümeleme Neden Zordur?

Hoca kümelemenin üç açıdan zor bir problem olduğunu anlattı.

Kümeler birbirine yakındır. İlk resimdeki kümeler birbirinden uzaktı. Böyle bir veride küme bulmak kolaydır. Ama gerçek hayattaki veri kümelerinde noktalar birbirine yakındır ve kümeleri birbirinden ayırmak zordur. Hoca her biri farklı renkte birçok kümenin iç içe durduğu bir resim gösterdi. Algoritmanın, farklı kümelerin noktaları birbirine bu kadar yakın olsa bile kümeleri doğru bulabilmesi gerekir.

Veri çok boyutludur. Örnekteki veri iki boyutluydu. Ama genellikle her veri noktasının 100 ya da daha fazla boyutu vardır. 10.000 boyutlu uygulamalar bile vardır. Yüksek boyutlu uzaylar farklı görünür. Kümeleri çıplak gözle göremeyiz. Ayrıca yüksek boyutta neredeyse bütün nokta çiftleri birbirine yaklaşık aynı uzaklıktadır. Hocanın açıklaması şuydu: İki nokta bazı boyutlarda birbirinden uzak, bazı boyutlarda birbirine yakın olabilir. Böyle düşününce yüksek boyutlu uzayda noktalar birbirine benzer uzaklıklarda kalır ve kümeleri ayırt etmek zorlaşır.

Veri tek bilgisayarın belleğine sığmaz. Bu dersin konusu açısından en önemli nokta budur. Hem her nokta yüksek boyutludur hem de veri miktarı o kadar büyüktür ki tek bir bilgisayarın belleğine sığmaz. Yöntemlerimiz birden çok bilgisayarda saklanan veriyi işleyebilmelidir.

Kısaca: İki boyutta kümeleme kolaydır. Az miktarda veriyi kümelemek de kolaydır. Ama çoğu durum böyle değildir.

3. Uygulama Örnekleri

3.1 Gök Cisimleri

Bir gök cismi kataloğu, her cismi 7 boyutlu ışınım (radiation) değerleriyle temsil eder. Her cisim 7 boyutlu bir vektördür ve toplam nokta sayısı 2 milyardır. Benzer cisimleri kümelemek isteriz. Bu kümeler galaksiler, yıldızlar vb. olabilir. Kümeleme algoritması bu veri kümesindeki bütün yıldızları ve galaksileri ayırt edebilmelidir.

> **Ders sonrası not:** Hoca slaytı okurken önce “2 million”, ardından “2 billion” dedi. Kaynak kitapta sayı 2 milyardır (2 billion); doğrusu 2 milyardır.

3.2 Müzik CD’leri

Müzik kategorilere ayrılır ve müşteriler birkaç kategoriyi tercih eder. Bir CD’yi, onu satın alan müşterilerin kümesiyle temsil ederiz. Böylece her CD bir müşteri vektörüyle gösterilir. Bu veri üzerinde kümeleme yaparsak, benzer CD’ler gruplanır ve buradan benzer müzik zevkine sahip müşteri grupları ortaya çıkar.

3.3 Dokümanlar

Aynı konudaki dokümanları gruplamak için de kümeleme kullanılır. Her doküman, içinde geçen kelimelerin kümesiyle temsil edilir. Aynı konudaki dokümanlarda benzer kelimeler geçer. Bu kelime vektörleri üzerinde kümeleme yapılarak aynı konudaki dokümanlar bulunur.

Aynı işi ters yönde de yapabiliriz. Bu kez dokümanları değil, kelimeleri gruplarız. Birçok dokümanda birlikte geçen (co-occur) kelimeler, dokümanlardaki varlıklarına göre aynı kümeye düşer.

4. Uzaklık Ölçüleri

Kümeleme tanımında bir uzaklık kavramından söz ettik. Kümeleri bulmak için farklı uzaklık ölçüleri kullanılabilir. Hoca derste Öklid, kosinüs, Jaccard ve düzenleme uzaklığını (edit distance) saydı, üçünü açıkladı.

Jaccard uzaklığı (Jaccard distance). Önceki bölümden bilinir. İki küme arasındaki benzerliği, kesişimin bileşime oranıyla ölçer:

$$\text{sim}_J(A, B) = \frac{|A \cap B|}{|A \cup B|}, \qquad d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

> **Ders sonrası not:** Hoca "paydada kesişim, paydada bileşim hesaplanır, bölümleri Jaccard uzaklığını verir" dedi. Doğrusu: Kesişim **paydada değil payda** (numerator), bileşim paydadadır. Bu oran Jaccard **benzerliğidir**; Jaccard uzaklığı 1 eksi bu oran-dır.

Öklid uzaklığı (Euclidean distance). Her noktanın sayısal değerlerle gösterilebildiği durumda noktalar arası uzaklığı verir. Hoca doküman örneğine döndü: Kelime vektöründe 1, kelimenin dokümanda geçtiğini, 0 geçmediğini gösterir. Vektöre bu sayısal değerleri koyarsak iki doküman arasındaki Öklid uzaklığını hesaplayabiliriz.

Kosinüs uzaklığı (cosine distance). Her veri temsili bir vektör olarak düşünülür. Vektörler arasındaki uzaklık, aralarındaki açının kosinüsüyle hesaplanır. Vektörler arası uzaklık arttıkça benzerlik azalır.

> **Ders sonrası not:** Hoca "iki vektör arasındaki kosinüs sıfırsa tam eşleşme vardır" dedi. Doğrusu: Tam eşleşmede açı sıfırdır, yani kosinüs **1**'dir ve kosinüs uzaklığı (açı) sıfırdır. Kosinüs sıfırsa vektörler birbirine diktir.

5. Kümeleme Yöntemlerinin İki Ana Ailesi

Kümeleme birçok farklı yoldan yapılabilir. Hoca yaygın bilinen iki kategoriye odaklandı.

Hiyerarşik kümeleme (hierarchical clustering). Algoritma çalışırken ek bir veri yapısı olan dendrogramı (dendrogram) kurar ve kümeleri bu yapıyı kullanarak bulur. İki alt türü vardır:

- **Birleştirici (agglomerative), aşağıdan yukarıya (bottom-up):** Başta her nokta bir kümedir. En yakın iki küme tekrar tekrar birleştirilir. Bu yinelemeli işlem sırasında dendrogram kurulur ve kümeler dendrogramdan belirlenir.
- **Bölücü (divisive), yukarıdan aşağıya (top-down):** Başta bütün veri tek kümedir. Ana küme, her nokta ayrı bir küme olana kadar yinelemeli olarak alt kümelere bölünür.

Yaygın olarak kullanılan birleştirici yöntemdir. Derste de ona odaklanıldı.

Nokta atama tabanlı kümeleme (point assignment). Başta bir küme kümesi tutulur. Küme sayısı başta belirlenir. Her yinelemede noktalar en yakın kümeye atanır. Bu da yinelemeli, nokta tabanlı bir algoritmadır.

6. Hiyerarşik Kümeleme

Hiyerarşik kümeleme bir **bellek içi (in-memory) algoritmadır**. Bütün verinin fiziksel belleğe sığdığı durumda kullanılır.

- **Avantajı:** Çok belirgin olmayan yapıları da bulabilir. Birbirine çok yakın duran yapıları iyi ayırır.
- **Dezavantajı:** Zaman karmaşıklığı yüksektir ve bütün verinin fiziksel bellekte olmasını ister.

> **Hocanın vurgusu:** Bu yüzden hiyerarşik kümelemeyi büyük ölçekli veriye doğrudan uygulayamayız. Yalnız kullanacağımız bütün veri belleğe sığıyorsa uygulayabiliriz. Bu algoritmalara bu nedenle bellek içi algoritmalar denir.

Birleştirici yaklaşımın ana işlemi şudur: En alttan başlanır, her nokta tek başına bir küme kabul edilir. Sonra en yakın iki küme, geriye tek büyük bir küme kalana kadar tekrar tekrar birleştirilir. Bu sırada dendrogram oluşur ve kümeler dendrogram kullanılarak çıkarılır.

6.1 Üç Temel Soru

Birleştirici kümelemeyi uygulamak için üç soruyu cevaplamak gerekir:

1. Çok noktalı bir küme nasıl temsil edilir?
2. İki kümenin yakınlığı nasıl belirlenir?
3. Kümeleri birleştirmeye ne zaman son verilir? Yani hangi anda "bu, son kümeleme sonucudur" deriz?

Birinci sorunun cevabı (Öklid uzayında): Kümeyi merkeziyle (centroid) temsil ederiz. Merkez, kümedeki noktaların ortalamasıdır. Bu ortalama boyut boyut alınır.

İkinci sorunun cevabı: Kümeler arası uzaklığı merkezler arası uzaklıkla ölçeriz. Bir kümeyi merkeziyle tanımladıysak, merkezi o kümenin referans noktası olarak kullanırız. İki kümenin uzaklığı, iki merkez arasındaki uzaklıktır.

Üçüncü soru örnekten sonra ele alındı (Bölüm 8).

7. Birleştirici Kümeleme Üzerine Adım Adım Örnek

Hoca slaytta altı noktalı bir örnek gösterdi. Sağ tarafta da algoritma ilerledikçe kurulan dendrogram vardı.

Başlangıç. Altı nokta var ve her biri ayrı bir kümedir.

1. yineleme. Bütün küme çiftleri arasındaki uzaklık hesaplanır. Altı küme olduğu için bütün çiftlere bakılır. En yakın çift $(1, 2)$ ile $(2, 1)$ çıkar. Bu ikisi tek bir kümede birleştirilir (slaytta mavi çember). Dendrogramda bu iki nokta en alttan birleştirilir.

Birleşen kümenin merkezi boyut boyut ortalamadır:

$$\left(\frac{1+2}{2},\ \frac{2+1}{2}\right) = (1.5,\ 1.5)$$

Bu yinelemeden sonra beş küme vardır. Biri iki noktalı, dördü tek noktalıdır.

> **Ders sonrası not:** Altyazıda hoca önce "x ekseninin ortalaması 1.1, y ekseninin ortalaması 1.1" diyor, hemen ardından merkezi "1.5 ve 1.5" olarak veriyor. Doğru değer $(1.5, 1.5)$'tir.

Dendrogramın eksenleri. X ekseni veri noktalarını, y ekseni zamanı (birleşme sırasını) gösterir. Zaman sıfırdan sonsuza doğru yukarı gider. Bir birleşme ne kadar sonra olursa dendrogramda o kadar yukarıda çizilir.

2. yineleme. Beş küme arasındaki bütün çift uzaklıkları yeniden hesaplanır. İki noktalı küme için uzaklık merkezinden ölçülür. En yakın çift $(4, 1)$ ile $(5, 0)$ çıkar ve birleştirilir. Bu birleşme birincisinden sonra olduğu için dendrogramda daha yukarıda çizilir. Artık dört küme vardır: ikisi tek noktalı, ikisi iki noktalı.

> **Ders sonrası not:** $(1, 2)$–$(2, 1)$ ve $(4, 1)$–$(5, 0)$ çiftlerinin uzaklığı eşittir: ikisi de $\sqrt{2} \approx 1.414$ (python ile doğrulandı). Yani 1. yinelemede iki çift eşit uzaklıktaydı. Hangisinin önce birleştirileceği bir eşitlik bozma kuralına bağlıdır; derste $(1, 2)$–$(2, 1)$ önce alındı.

3. yineleme. Dört küme arasındaki çift uzaklıklar hesaplanır. Tek noktalı bir küme ile iki noktalı $\{(1, 2), (2, 1)\}$ kümesi en yakın çıkar ve birleştirilir. Dendrogramda bu birleşme önceki ikisinin üstünde çizilir.

4. yineleme. Artık üç küme vardır: bir tek noktalı, bir iki noktalı, bir üç noktalı. Üç noktalı kümenin merkezi hoca tarafından $(1, 1)$ olarak verildi.

Altyazıda üç noktalı kümeye katılan noktanın koordinatı okunmuyor. Merkezin $(1, 1)$ çıkması için bu noktanın $(0, 0)$ olması gerekir (ara adım eklendi):

$$\frac{1+2+x}{3} = 1 \Rightarrow x = 0, \qquad \frac{2+1+y}{3} = 1 \Rightarrow y = 0$$

Kontrol: $\left(\frac{1+2+0}{3}, \frac{2+1+0}{3}\right) = (1, 1)$. Ayrıca $(0, 0)$ noktasının $(1.5, 1.5)$ merkezine uzaklığı $\sqrt{4.5} \approx 2.12$'dir (python ile doğrulandı).

Son adımlar. Çift uzaklıklar yeniden hesaplanır. Tek noktalı küme, en yakın olduğu kümeyle birleştirilir. Sonunda her şey tek bir kümede birleşir. Bütün bu sürecin amacı dendrogramı kurmaktır.

7.1 Soru: Merkez Bütün Veriyle mi Hesaplanır?

Soru: Algoritma merkezi veritabanındaki bütün noktalar için mi hesaplıyor ve en kısa merkez uzaklığına göre mi birleştirilecek kümeye karar veriyor?

Cevap: Hayır. Merkez hesaplanırken yalnız o kümeye ait noktalar kullanılır. Örnekte ilk birleşmede iki nokta vardı, veri kümesinde ise toplam altı nokta. Merkez için bütün veri değil, yalnız aynı kümedeki iki nokta kullanıldı. Sonraki yinelemede yeni bir küme oluşunca onun merkezi de yalnız o kümenin noktalarıyla hesaplandı.

Hoca buna bir gözlem ekledi: Aşağıdan yukarı çıktıkça kümelerdeki nokta sayısı artar. Örneğin geriye üç küme kaldığında neredeyse bütün noktalar bu kümelerin içindedir. Kümelerdeki toplam nokta sayısı çok büyüyebilir ve merkezi bulmak zaman alır. Geriye iki küme kaldığında ise aralarındaki uzaklığı hesaplamanın bir anlamı yoktur, çünkü birleştirilecek tek çift onlardır.

Hoca, merkezi her seferinde bütün noktaların ortalamasını yeniden alarak hesaplamak yerine bir özet bilgi tutarak kısa yoldan hesaplamanın yolunu dersin ilerisinde göstereceğini söyledi. Bu yol BFR algoritmasındaki özet istatistiklerdir (Bölüm 16).

8. Ne Zaman Durmalı? Küme Kalitesi

Hoca şu soruyu sordu: Amacımız belirgin kümeleri bulmaksa, dendrogramı neden en tepeye kadar kurmak zorundayız? Ne zaman duracağımızı nasıl bileceğiz?

Kaliteye göre durma. Her birleştirmeden sonra küme kalitesi (clustering quality) hesaplanır. Örneğin iki küme birleşir, o anda kalite hesaplanır. Bir sonraki birleşmede yine hesaplanır. Bu değerler izlenir. Kalite düşmeye başladığında algoritma durdurulur. O nokta, hiyerarşik kümelemeyle ulaşılan en iyi kümelemedir. Bu yöntem veri hakkında ön bilgi istemez.

Küme sayısına göre durma. Başta veride örneğin üç küme olduğu biliniyorsa, k farklı küme bulununca birleştirme durdurulur. Bu yöntem veri hakkında ön bilgi ister: Kaç küme olduğunu bilmek gerekir.

8.1 Sıkılık (Cohesion)

Küme kalitesi ölçülerinden biri sıkılıktır (cohesion). Sıkılık, kümelerin ne kadar derli toplu olduğunu, yani küme içindeki noktaların birbirine ne kadar yakın olduğunu söyler. Hoca sıkılığı hesaplamak için üç yaklaşım saydı:

- **Çap (diameter):** Birleşmiş kümedeki noktalar arasındaki en büyük uzaklık.
- **Yarıçap (radius):** Bir noktanın kümeye olan en büyük uzaklığı. Çap ya da yarıçap doğrudan sıkılık değeri olarak kullanılabilir.
- **Yoğunluk tabanlı (density-based):** Kümedeki nokta sayısı bulunur ve çapa ya da yarıçapa bölünür.

  Ders sonrası not: Altyazıda yarıçap "bir noktanın birbirinden en büyük uzaklığı" diye geçiyor. Kitaptaki tanım büyük olasılıkla şudur: Yarıçap, kümedeki bir noktanın merkeze (ya da clustroid'e) olan en büyük uzaklığıdır.

Sıkılık, küme kalitesi ölçülerinden yalnızca biridir. Başka ölçüler de vardır.

9. Öklid Dışı Uzay ve Clustroid

Merkez yalnız Öklid uzayında (Euclidean space) bulunabilir. Öklid uzayı, her boyutun değerinin sayısal olduğu uzaydır. Noktaların öznitelikleri sayısal değilse merkezi hesaplayamayız. O zaman kümeyi temsil edecek başka bir nokta bulmamız gerekir.

Bu noktaya **clustroid** denir. Clustroid, kümedeki noktalardan diğer noktalara en yakın olandır. Bu noktayı kümenin temsilcisi olarak kullanırız.

Hocanın örneğinde sayısal olmayan uzayda üç nokta vardı. Öznitelikler sayısal olsaydı ortalamayı alıp merkez diyebilirdik. Bu mümkün olmadığı için noktalardan biri temsilci seçilir. Diğerlerine en yakın olan, ortadaki nokta seçilir ve clustroid adını alır. Bundan sonra bu kümeye uzaklık hesaplanırken clustroid kullanılır.

Kümeler arası yakınlık: Kümeler arası uzaklık hesaplanırken clustroid sanki merkezmiş gibi kullanılır. Sayısal olmayan öznitelikler arasındaki uzaklığı ölçen ölçüler de vardır, yani Öklid dışı uzayda da uzaklık ölçüleri kullanılabilir.

"Diğer noktalara en yakın" ifadesi farklı biçimlerde tanımlanabilir. Clustroid şu ölçütlerden biriyle seçilir:

- Diğer noktalara en büyük uzaklığı en küçük olan nokta (smallest maximum distance).
- Diğer noktalara ortalama uzaklığı en küçük olan nokta (smallest average distance).
- Diğer noktalara uzaklıklarının kareleri toplamı en küçük olan nokta (smallest sum of squares of distances).

Durma koşulu Öklid dışı uzayda da aynıdır: Her birleştirmeden sonra sıkılık gibi bir kalite ölçüsü hesaplanır ve düşmeye başlayınca durulur. Ya da k biliniyorsa k küme kalınca durulur.

10. Hiyerarşik Kümelemenin Karmaşıklığı

Her yinelemede bütün küme çiftleri arasındaki uzaklık hesaplanır. Bu, en kötü durumda n nokta için $O(n^2)$ iştir. En kötü durumda ağacın yüksekliği, yani yineleme sayısı da n olabilir. Bu ikisinin çarpımı toplam karmaşıklığı verir:

$$O(n) \cdot O(n^2) = O(n^3)$$

n^3 çok maliyetli bir karmaşıklıktır. Büyük ölçekli veride toplam çalışma süresi çok yüksek olur. Algoritmanın bellek içi algoritma olarak bilinmesinin ve büyük veriye ölçeklenmemesinin sebebi budur.

Dikkatli bir uygulamayla karmaşıklık $O(n^2 \log n)$'ye indirilebilir. Ama bu da büyük veri için hâlâ yüksektir. Bu yüzden algoritma yalnız küçük ve belleğe sığan veri kümelerinde çalışır.

> **Ders sonrası not:** Altyazıda bu ifade "n log n çarpı n kare" olarak geçiyor. Bu $O(n^3 \log n)$ demek olur, ki $O(n^3)$'ten bile kötüdür. Doğrusu $O(n^2 \log n)$'dir.
>
> **Ek bilgi (derste anlatılmadı):** $O(n^2 \log n)$ sonucuna büyük olasılıkla şöyle ulaşılır: Başta bütün çift uzaklıkları ($O(n^2)$) hesaplanıp bir öncelik kuyruğuna (priority queue) konur. Her birleştirmede kuyruktan en yakın çift alınır, birleşen kümelerle ilgili çiftler silinip yeni kümenin uzaklıkları eklenir. Her birleştirme $O(n \log n)$ sürer ve en fazla n birleştirme olur.

11. K-Means Algoritması

Molanın ardından hoca ikinci kategoriye, atama tabanlı kümelemeye geçti. Bu ailenin temsilcisi K-means algoritmasıdır.

Varsayım ve girdi:

- K-means **Öklid uzayını varsayar**. Yalnız bütün özniteliklerin sayısal olduğu verilerde kullanılabilir.
- Başlamadan önce küme sayısı k verilmelidir. Bu yüzden veride kaç küme olduğu hakkında ön bilgi gerekir.

Algoritma önce her küme için veri kümesinden bir temsilci nokta seçer. Sonra kalan noktaları bu kümelere atar ve bunu yinelemeli olarak sürdürür.

Adımlar:

1. k başlangıç noktası seçilir. Bunlar ilk merkezlerdir.
2. Her nokta, şu anki merkezi en yakın olan kümeye yerleştirilir.
3. Bütün noktalar tarandıktan sonra k kümenin merkezlerinin konumu güncellenir.
4. Döngünün başına dönülür. Bütün noktalar yeniden en yakın merkeze atanır. Bazen noktalar kümeler arasında yer değiştirir.
5. Adım 2 ve 3, algoritma yakınsayana (converge) kadar tekrarlanır. Yakınsama koşulu: Noktalar kümeler arasında yer değiştirmez ve merkezler sabitlenir.

11.1 Örnek: 11 Nokta, $k = 2$

Hoca slaytta 11 noktalı bir örnek gösterdi. Ön bilgiyle veride iki küme olduğunu bildiğimizi varsayıp $k = 2$ verdi.

- **Başlangıç:** Algoritma iki noktayı başlangıç merkezi olarak seçer.
- **1. tur:** Bütün noktalar en yakın merkeze atanır. Bu turda bir kümeye 3 nokta, diğerine kalan 8 nokta düşer. Sonra iki kümenin merkezi yeniden hesaplanır.
- **2. tur:** Bütün noktalar, ilk noktadan son noktaya kadar, yeni merkezlere göre yeniden atanır. Bu kez 3 nokta bir merkeze, kalanlar diğerine en yakın çıkar. Kümelerin üyeleri bilindiği için merkezler yeniden hesaplanır.
- **3. tur:** Noktaların merkezlere uzaklıkları yeniden hesaplanır ve atama yapılır. Merkezler tekrar hesaplanır.
- **4. tur:** Merkezler değişmez, noktalar kümeler arasında yer değiştirmez. Algoritma yakınsar ve durur.

12. k Değerini Seçmek

Hoca şu soruyu sordu: Veri hakkında ön bilgim yoksa, veride kaç küme olduğunu bilmiyorsam ne yapacağım?

Cevap: Farklı k değerleri denenir. $k = 2$'den başlanır, sonra 3, 4 ve örneğin 10, 11'e kadar çıkılır. Her k için farklı bir kümeleme sonucu bulunur ve her sonuç için sıkılık ya da başka bir küme kalitesi ölçüsü hesaplanır. En iyi kaliteyi veren k seçilir.

Örnek. Hocanın slaytındaki veride açıkça üç küme vardı. Ama başta ön bilgimiz yok ve veriyi görselleştirme imkânımız yok diye düşünüyoruz.

- $k = 2$**:** Kümeler birbirinden uzak noktaları da içine alır. Kümelerden birinin çapı çok büyüktür. Bu sonuç için küme kalitesi kötüdür.
- $k = 3$**:** K-means üç belirgin kümeyi bulur. Her kümenin çapı küçüktür. Kalite $k = 2$'ye göre çok daha iyidir.
- $k = 4$ **ve üstü:** Küme sayısı arttıkça ölçü yine azalmaya devam eder, ama artık az miktarda.

  Ders sonrası not: Hoca bir yerde "$k = 2$ için sıkılık kalitesi küçük olur", hemen ardından "$k = 3$ için sıkılık ölçüsü küçük olur" dedi. Burada "sıkılık değeri" olarak çap ya da merkeze ortalama uzaklık kastediliyor; bunun küçük olması kalitenin **iyi** olduğunu gösterir. Yani $k = 2$'de çap büyük (kalite kötü), $k = 3$'te çap küçük (kalite iyi).

12.1 Merkeze Ortalama Uzaklık Grafiği

Bu seçimi yapmanın bir yolu bir grafik çizmektir:

- X ekseni: k değeri.
- Y ekseni: merkeze ortalama uzaklık (average distance to centroid). Her küme için noktaların merkezine ortalama uzaklığı bulunur, sonra bütün kümelerin ortalaması alınır.

k, 2'den başlayıp örneğin 10, 11'e kadar artarken her değer için bu ortalama hesaplanır ve grafiğe konur. k bir artınca ortalama uzaklık önce sert biçimde düşer. Bir artış daha, yine

sert düşüş. Bir noktadan sonra k bir artınca ortalama uzaklık hâlâ düşer, ama az miktarda. **Düşüşün belirgin biçimde yavaşladığı bu k değeri seçilir** ve kümeleme bu değerle yapılır.

> **Ek bilgi (derste anlatılmadı):** Bu yönteme yaygın olarak "dirsek yöntemi" (elbow method) denir, çünkü grafik düşüşün yavaşladığı noktada bir dirsek gibi kırılır. k'yi en küçük ortalama uzaklığı verene göre seçmek doğru olmaz: k nokta sayısına eşit olursa her nokta kendi merkezi olur ve ortalama uzaklık sıfıra iner.

13. Başlangıç Noktalarını Seçmek

k belirlendikten sonra karar verilmesi gereken ikinci şey, k kümenin başlangıç temsilcilerinin nasıl seçileceğidir. Başlangıç noktaları önemlidir, çünkü atama tabanlı algoritmanın çalışma süresini etkiler. Hoca iki yöntem anlattı.

1. Örnekleme (sampling). Veri kümesinden temsil edici, belleğe sığan bir örnek seçilir. Örnek, hiyerarşik kümeleme algoritmalarından biriyle kümelenir. Hiyerarşik kümeleme k değerini istemez; küme sayısını algoritma kendisi verir. Böylece hem k hem de örnekteki kümeler bulunur. Örnekteki her kümenin merkezi hesaplanır. Her kümeden merkeze en yakın nokta seçilir ve K-means bu noktalarla başlatılır.

2. Dağınık noktalar (dispersed set of points). Örneğin $k = 3$ olsun.

- İlk küme için veri kümesinden rastgele bir nokta seçilir.
- İkinci başlangıç noktası için rastgele seçilen ilk noktanın bütün noktalara uzaklığı hesaplanır. İlk noktadan en uzak olan seçilir.
- Üçüncü nokta için bütün noktaların birinci ve ikinci noktaya uzaklığı hesaplanır. İkisinden de uzak olan seçilir.
- k nokta olana kadar bu tekrarlanır.

> **Ders sonrası not:** "Seçilmiş noktaların hepsinden uzak" ifadesinin kesin ölçütü büyük olasılıkla şudur: Aday noktanın seçilmiş noktalara olan **en küçük** uzaklığı hesaplanır ve bu değeri **en büyük** olan nokta seçilir.

14. K-Means'in Karmaşıklığı

K-means'in her turunda her nokta tam bir kez incelenir ve en yakın merkez bulunur. Bu yüzden bir tur N ile orantılıdır. Ama her nokta mevcut k merkezin hepsiyle karşılaştırılır. Bu nedenle bir turun karmaşıklığı:

$$O(kN)$$

Yakınsamak için gereken tur sayısı oldukça büyük olabilir. Toplam süre, bir turun maliyetinin tur sayısıyla çarpımıdır.

Hoca buradan bir soruya geçti: Veriyi tek geçişte (single pass) kümeleyebilir miyiz? K-means'i veri üzerinden tek bir geçişle nasıl uygulayabiliriz? Bu sorunun cevabı BFR algoritmasıdır.

Hoca, şimdiye kadar anlatılan algoritmaların bilgiye erişim (information retrieval), makine öğrenmesi gibi başka derslerde de görülebileceğini, bu dersin farkının bu algoritmaların büyük veri sürümlerini görmek olduğunu söyledi.

15. BFR Algoritması

BFR (Bradley-Fayyad-Reina) algoritması, K-means'in çok büyük, fiziksel belleğe sığmayan veri kümeleri için tasarlanmış bir türüdür.

Varsayımlar:

- K-means gibi yalnız Öklid uzayında çalışır. Özniteliklerin değerleri sayısal olmalıdır.
- **Ek varsayım:** Her kümede, her boyutta veri merkez etrafında normal dağılır (normally distributed around the centroid).

Hoca bunu iki boyutlu bir resimle açıkladı. Bir kümenin noktalarının yalnız x değerlerini alıp bir normal dağılıma koyarsak dağılıma tam oturur. Noktaların çoğu ortada bir yerdedir, kenarlara gittikçe nokta sayısı azalır. Y ekseninde de aynı davranış görülür. Resimdeki diğer kümelerde de durum aynıdır. Bu tür bir veri, BFR için kusursuz bir veridir.

15.1 Normal Dağılım Hatırlatması

Bir veri kümesinin ortalaması $\bar{x}$ ve standart sapması σ olsun.

- Ortalamadan her iki yönde 1σ gidilirse verinin yaklaşık %68'i bu aralıktadır.
- 2σ gidilirse verinin yaklaşık %95'i bu aralıktadır.
- 3σ gidilirse verinin yaklaşık %99.7'si bu aralıktadır.

  Ders sonrası not: Hoca 3σ için "sanırım %98" dedi. Doğru değer yaklaşık **%99.7**'dir. Python ile hesaplandı: 1σ için 0.6827, 2σ için 0.9545, 3σ için 0.9973.

Veri her kümede, her boyutta ortalama etrafında böyle dağılıyorsa BFR büyük ölçekli verideki kümeleri bulabilir.

15.2 Algoritmanın Akışı

Hocanın örneği: Elimizde 1 terabaytlık veri ve yalnız 16 gigabaytlık fiziksel bellek var. Bu veriyi kümelemek istiyoruz. Veri küçük olsaydı, yüksek kaliteli sonuç veren hiyerarşik kümeleme gibi bellek içi bir algoritma kullanırdık. Ama terabaytlarca veri küçük bir belleğe sığmaz. BFR şunu önerir:

1. Veri, her biri fiziksel belleğe sığan parçalara (segment) bölünür. Noktalar diskten belleğe parça parça okunur.
2. İlk parça belleğe alınır. Bellek içi algoritmalardan biri uygulanarak başlangıç kümeleri bulunur. Bu iş için K-means de kullanılabilir, ama yaygın yol hiyerarşik kümelemedir, çünkü k değerini istemez.
3. Bulunan kümeler için basit istatistik bilgileri hesaplanır.
4. Noktalar üç gruba ayrılır (aşağıda).
5. Sonraki parçalar sırayla belleğe alınır ve noktaları kümelere atanır.
6. Bütün parçalar bittikten sonra bellekte kalan noktalar en yakın kümelere katılır.

15.3 Üç Küme Türü: DS, CS, RS

Hiyerarşik kümeleme ilk parçaya uygulandığında bize kümeleri ve bu kümelerin hiçbirine girmeyen noktaları verir. Bunlardan üç tür küme çıkarılır:

- **Discard set (DS):** Büyük kümeler. Bu kümelere giren bütün noktalara discard set denir. Bu noktalar fiziksel bellekte tutulmaz; adı da buradan gelir. Her büyük küme için yalnız üç istatistik bilgisi tutulur.
- **Compression set (CS):** Birbirine yakın, ama büyük kümelere girmeyen noktaların oluşturduğu küçük kümeler. Bu küçük kümelerin bütün noktaları bellekte tutulur. Onlarla bir şey yapılmaz, yalnız izlenir.
- **Retained set (RS):** Hiçbir kümeye girmeyen tek tek noktalar. Bunlar da bellekte tutulur.

Hoca bu kısmı Türkçe olarak da tekrarladı: İlk parça için bellek içi algoritmalardan biriyle kümeler belirlenir. Büyük kümelerin içinde kalan bütün noktalara discard set denir. Bu noktalar bellekte tutulmaz. Onların yerine bu kümeyi ifade eden üç istatistik tutulur: kümedeki nokta sayısı N, SUM vektörü ve SUMSQ vektörü.

16. BFR'de Özet İstatistikler: N, SUM, SUMSQ

d boyutlu veride her DS kümesi için şu üç bilgi tutulur:

- **N:** Kümedeki nokta sayısı. Kümede 100 nokta varsa $N = 100$, 50 nokta varsa $N = 50$.
- **SUM:** d boyutlu bir vektör. i. bileşeni, kümedeki bütün noktaların i. koordinatlarının toplamıdır:

$$\text{SUM}_i = \sum_{p \in C} p_i$$

- **SUMSQ:** Yine d boyutlu bir vektör. i. bileşeni, noktaların i. koordinatlarının karelerinin toplamıdır:

$$\text{SUMSQ}_i = \sum_{p \in C} p_i^2$$

Hocanın örneği: Kümede 100 nokta olsun ve boyut 5 olsun. Bu durumda SUM vektörünün boyutu da 5'tir.

16.1 Bir Küme İçin Kaç Sayı Tutulur?

Hoca sordu: Discard set'teki her küme için kaç tane sayı tutuyorum? Cevap $2d + 1$'dir.

- SUM vektörünün boyutu d'dir.
- SUMSQ vektörünün boyutu d'dir.
- N için tek bir sayı tutulur.

Toplam: $d + d + 1 = 2d + 1$ sayı. Bu, bir kümeyi bellekte temsil etmek için gereken alanın tamamıdır. Amaç, kümedeki noktaları bellekte tutmamak, yalnız temsil edici istatistikleri tutmaktır.

16.2 Neden Bu İstatistikler? Merkez ve Standart Sapma

SUM ve SUMSQ'yu hesaplamamızın sebebi, ortalamayı ve standart sapmayı bulmaktır.

Merkez: SUM vektörünün her boyutu N'ye bölünür. Merkez d boyutlu olduğu için bölme boyut boyut yapılır:

$$c_i = \frac{\text{SUM}_i}{N}$$

Standart sapma: i. boyuttaki varyans ve standart sapma şöyle bulunur:

$$\sigma_i^2 = \frac{\text{SUMSQ}_i}{N} - \left(\frac{\text{SUM}_i}{N}\right)^2, \qquad \sigma_i = \sqrt{\frac{\text{SUMSQ}_i}{N} - \left(\frac{\text{SUM}_i}{N}\right)^2}$$

Bu hesap için gereken her şey zaten elimizdedir: SUMSQ her adımda güncellenir, SUM her adımda güncellenir, N de tutulan bilgilerden biridir. Böylece ilk hiyerarşik kümelemeden sonra her küme için hem merkez hem standart sapma her adımda hesaplanabilir. Standart sapma, kümedeki noktaların nasıl dağıldığını söyler.

> **Ders sonrası not:** Bu formüller python (numpy) ile küçük bir girdi üzerinde doğrulandı: N, SUM ve SUMSQ'dan hesaplanan merkez ve varyans, noktaların doğrudan ortalaması ve (popülasyon) varyansıyla aynı çıktı. Yeni bir nokta eklenip istatistikler güncellendiğinde de sonuç doğrudan hesapla aynıydı.

16.3 İkinci Parça ve Sonrası

İlk adımdan sonra her kümenin merkezi ve standart sapması bilinir. Bu kümelere girmeyen noktalar bellekte tutulur. Tutulmayan yalnız büyük kümelerin içindeki noktalardır.

Sonra ikinci parça belleğe alınır. Bu kez parçadaki noktalar mevcut kümelere atanır:

- Noktanın merkezlere uzaklığına bakılır. Uzaklık belli bir eşiğin altındaysa nokta o kümeye aittir. Nokta kümeye eklenir ve atılır (discard), yani bellekte tutulmaz.
- Nokta hiçbir kümeye yeterince yakın değilse bellekte tutulur.

Bu üçüncü, dördüncü, beşinci parça için de tekrarlanır. Bir DS kümesine her yeni nokta eklendiğinde istatistikler güncellenir:

- N bir artırılır.
- Noktanın koordinatları SUM vektörüne eklenir.
- Koordinatların kareleri SUMSQ vektörüne eklenir.

Yani discard set, merkeze atılabilecek kadar yakın olan ve ana kümelere ait noktalardır. CS ve RS ise bellekte tutulması gereken noktalardır.

Bu noktada iki önemli soru vardır:

1. Bir noktanın bir kümeye eklenecek kadar yakın olduğuna nasıl karar veririz?
2. İki CS alt kümesinin birleştirilmeyi hak ettiğine nasıl karar veririz?

17. Mahalanobis Uzaklığı ve Eşik Seçimi

BFR, bir noktanın bir kümeye ait olup olmadığına Mahalanobis uzaklığı (Mahalanobis distance) ile karar verir. Bu uzaklık, noktanın şu an en yakın merkeze ait olma olasılığına (likelihood) dayanır.

17.1 Hesap

Kümeler ve her kümenin merkezi bulunmuş olsun. Her küme için standart sapmalar da hesaplanmış olsun. Bunlar, her küme için tutulan üç istatistikten (Bölüm 16) bulunur.

- Nokta: $p = (p_1, p_2, \dots, p_d)$.
- C kümesinin merkezi: $c = (c_1, c_2, \dots, c_d)$.
- C kümesinin her boyuttaki standart sapması: $\sigma_1, \sigma_2, \dots, \sigma_d$.

Her boyut için noktanın değeri ile merkezin o boyuttaki değeri arasındaki fark alınır ve o boyutun standart sapmasına bölünür. Bu, o boyuttaki normalleştirilmiş uzaklıktır:

$$y_i = \frac{p_i - c_i}{\sigma_i}$$

Bu y vektörü, p noktası ile C kümesi arasındaki normalleştirilmiş uzaklığı verir. Mahalanobis uzaklığı, y'nin bileşenlerinin karelerinin toplamının karekökü'dür:

$$d(p, c) = \sqrt{\sum_{i=1}^{d} y_i^2} = \sqrt{\sum_{i=1}^{d} \left(\frac{p_i - c_i}{\sigma_i} \right)^2}$$

Bu hesap, p noktası için her kümeyle yapılır: küme 1, küme 2, küme 3. Sonra uzaklığın bir eşiğin altında olup olmadığına bakılır. C kümesi için eşiğin altındaysa nokta C'ye, başka bir küme için eşiğin altındaysa o kümeye aittir.

17.2 Eşik Nasıl Seçilir?

Hoca eşiği şöyle türetti. y vektörünün bütün değerlerinin 1 olduğunu varsayalım. Bu, p ile merkez arasındaki farkın her boyutta tam bir standart sapma olduğu anlamına gelir: Her boyutta fark σ_i ise $y_i = \sigma_i / \sigma_i = 1$ olur.

Bu durumda kökün içindeki toplam, d tane 1'in toplamıdır, yani d'dir:

$$d(p, c) = \sqrt{\underbrace{1^2 + 1^2 + \cdots + 1^2}_{d \text{ tane}}} = \sqrt{d}$$

Aynı mantıkla:

- Nokta her boyutta merkezden 1σ uzaksa uzaklık $\sqrt{d}$.

- Her boyutta 2σ uzaksa her $y_i = 2$, uzaklık $\sqrt{4d} = 2\sqrt{d}$.
- Her boyutta 3σ uzaksa her $y_i = 3$, uzaklık $\sqrt{9d} = 3\sqrt{d}$.

  Ders sonrası not: Bu değerler python ile $d = 3$ için doğrulandı: Merkezden her boyutta 1σ, 2σ, 3σ uzaktaki noktaların Mahalanobis uzaklığı sırasıyla $1.732 = \sqrt{3}$, $3.464 = 2\sqrt{3}$ ve $5.196 = 3\sqrt{3}$ çıktı.

Boyut sayısı d başta bilindiği için bu değerler eşik olarak kullanılabilir. Gelen her nokta için Mahalanobis uzaklığı hesaplanır ve $\sqrt{d}$, $2\sqrt{d}$ ya da $3\sqrt{d}$ ile karşılaştırılır. Bunlardan herhangi biri eşik seçilebilir. Örneğin eşik $3\sqrt{d}$ seçilirse, hesaplanan uzaklık bu değerden küçük olduğunda nokta kümeye kabul edilir.

17.3 Akışın Tekrarı

Hoca akışı bir kez daha özetledi. Büyük kümelere ait noktalar bellekte tutulmadı. Yalnız bu kümelere girmeyen noktalar CS ve RS olarak tutuldu. Kümelere ait noktalar DS oldu. İkinci parça belleğe alındı. Parçadaki her noktanın kümelerin merkezlerine Mahalanobis uzaklığı hesaplandı. Uzaklık eşiğin altındaysa nokta o kümeye eklendi. Değilse diğer merkezlerle karşılaştırıldı. Hiçbirine ait değilse bellekte tutuldu. Bir kümeye aitse N, SUM ve SUMSQ güncellendi ve nokta atıldı.

Gerçek noktalar yerine istatistik bilgisi tutmanın sebebi, verinin terabaytlarca, belleğin ise gigabaytlarca olmasıdır. Bütün noktalar belleğe sığmaz. Her adımda kümelerin istatistikleri güncellendiği için her kümeyle ilgili hesaplar için gereken bilgi hep elimizdedir ve bellek gereksiz yere kullanılmaz.

18. CS Kümelerinin Birleştirilmesi ve Algoritmanın Sonu

İkinci soru, iki CS alt kümesinin birleştirilip birleştirilemeyeceğidir. DS kümelerinin dışında bellekte tutulan noktalar vardı. Bunların içinde birlikte gruplanabilen noktalar da vardı. Bu küçük kümelere CS kümeleri denir.

Her yinelemede, yani belleğe her yeni parça alındığında, iki küçük kümenin birleştirilip birleştirilemeyeceği de hesaplanır. Bunun için bellek içi kümeleme algoritmalarından biri kullanılır ve kümeler merkezlerine bakılarak birleştirilir.

Bütün parçalar işlendikten sonra, bellekte tutulan ve DS'ye ait olmayan bütün CS kümeleri ve noktalar en yakın kümelere katılır.

> **Ek bilgi (derste anlatılmadı):** Kitapta CS kümeleri de büyük olasılıkla DS gibi N, SUM ve SUMSQ ile özetlenir ve iki CS kümesinin birleşip birleşmeyeceğine, birleşik kümenin varyansı bir eşiğin altında kalıyor mu diye bakılarak karar verilir. Bu istatistikler toplanabilir olduğu için birleşik kümenin istatistikleri iki kümenin N, SUM ve SUMSQ değerlerinin toplamıdır.

19. CURE Algoritması

Hoca dersi CURE algoritmasıyla bitirdi.

19.1 Neden CURE?

CURE, BFR'nin varsayımını kaldırır. BFR, kümelerin her boyutta normal dağılmasını istiyordu. Örneğin iki boyutta x için de y için de her küme normal dağılmalıydı. Böyle bir veri BFR ile işlenebilir.

Hoca buna uymayan veri kümeleri gösterdi. Bir resimde noktalar x ve y eksenlerinde normal dağılmıyordu. Başka bir resimde de her kümenin noktaları her boyutta normal dağılmıyordu. Büyük ölçekli bütün veriler boyut boyut normal dağılmaz. Böyle verilerde CURE kullanılabilir.

Özellikleri:

- CURE de Öklid uzayını varsayar, çünkü ortalama ve merkez hesaplaması gerekir.
- Ama kümelerin her biçimde olmasına izin verir. Farklı biçimli kümeler de bulunabilir.

19.2 Örnek Veri

Hoca basit tutmak için iki boyutlu bir veri gösterdi. Veride iki ana küme vardı ve bir kısımları birbirine geçmişti (collided). Soru, CURE'un böyle kümeleri bulmamıza yardım edip edemeyeceğiydi.

19.3 Birinci Aşama

1. Belleğe sığan rastgele bir örnek seçilir. Yine terabayt ölçeğinde bir veri ve gigabayt ölçeğinde bir bellek düşünülür. Diskteki büyük veriden yalnız rastgele seçilmiş bir örnek belleğe alınır.
2. Örnek noktalar, hiyerarşik kümeleme gibi bellek içi bir algoritmayla kümelenir ve başlangıç kümeleri bulunur.
3. Her kümeden, birbirinden olabildiğince uzak k temsilci nokta seçilir. k örneğin 4 ya da 3 olabilir.
4. Her temsilci nokta, kümenin merkezine doğru sabit bir oranda kaydırılır.

Hocanın örneğinde hiyerarşik kümeleme örnek üzerinde üç küme buldu. Sonuç, gerçek yapıdaki kadar kusursuz değildi. Ama iki küme tamamen tek türden noktalardan oluşuyordu, üçüncüsünün ise büyük kısmı tek türdendi ve diğer türden de noktalar içeriyordu. Verinin gerçek dağılımını bilmeden bakıldığında bu iyi bir kümeleme sonucu sayılabilir.

Sonra her kümeden dört temsilci nokta seçildi. Temsilciler olabildiğince dağınık seçilir: Önce biri rastgele seçilir. İkincisi birinciden uzak olmalıdır. Üçüncüsü ikisinden de uzak olmalıdır. Dördüncüsü hepsinden uzak olmalıdır.

Ardından her temsilci, merkeze doğru belli bir yüzde kadar kaydırılır. Örnekte bu oran %20 seçildi. Bu işlem her kümeye uygulanır. Sonunda her kümenin dört temsilcisi olur.

19.4 İkinci Aşama

Bütün veri kümesi yeniden taranır. Her p noktası, temsilci noktalardan en yakın olanın kümesine atanır. Tanım: p, bütün kümelerin bütün temsilci noktaları arasında kendisine en yakın temsilciye sahip olan kümeye yerleştirilir.

Örnekte üç küme ve her birinde dört temsilci vardır:

$$3 \times 4 = 12 \text{ temsilci nokta}$$

Veri kümesindeki her nokta için bu 12 temsilciye olan uzaklık hesaplanır. Nokta, en yakın temsilcinin ait olduğu kümeye atanır. Bütün veri tarandığında her nokta en yakın kümeye yerleştirilmiş olur.

> **Ek bilgi (derste anlatılmadı):** Temsilcileri merkeze doğru kaydırmanın amacı büyük olasılıkla, kümenin kenarındaki uç noktaların etkisini azaltmaktır. Kümeyi tek bir merkez yerine birden çok temsilci noktayla göstermek de uzun, kıvrımlı ya da birbirine geçmiş kümelerin biçimini yakalamayı sağlar.

Örnek Sorular

Bu derste hazır kare ya da video yoktur. Sorular yalnız derste sözlü olarak sorulan sorulardan alınmıştır.

Soru 1 (derste sözlü, 00:42:47 ve 00:45:15): Altı noktalı birleştirici kümeleme örneğinde başta her nokta bir kümedir. 1. yinelemeden sonra kaç küme vardır? 2. yinelemeden sonra kaç küme vardır? 3. yinelemeden sonra kaç küme vardır?

> **Çözüm:** Her birleştirme küme sayısını bir azaltır. 1. yinelemeden sonra $6 - 1 = 5$ küme (biri iki noktalı). 2. yinelemeden sonra 4 küme (ikisi tek noktalı, ikisi iki noktalı). 3. yinelemeden sonra 3 küme (bir tek noktalı, bir iki noktalı, bir üç noktalı).

Soru 2 (derste sözlü, 00:48:08): Birleştirici kümelemede merkez, veritabanındaki bütün noktalar için mi hesaplanır ve birleştirilecek küme en kısa merkez uzaklığına göre mi seçilir?

Çözüm: Hayır. Merkez yalnız o kümeye ait noktalarla hesaplanır. Örnekte altı noktanın hepsi değil, yalnız birleşen kümedeki noktalar kullanıldı. Birleştirilecek çift ise kümelerin merkezleri arasındaki uzaklığa göre seçilir.

Soru 3 (derste sözlü, 00:52:22): Amacımız belirgin kümeleri bulmaksa dendrogramı neden en tepeye kadar kurmalıyız? Ne zaman duracağımızı nasıl biliriz?

Çözüm: Her birleştirmeden sonra küme kalitesi (örneğin sıkılık) hesaplanır ve kalite düşmeye başladığında durulur. Veride kaç küme olduğu biliniyorsa k küme kalınca da durulabilir; bu ön bilgi ister.

Soru 4 (derste sözlü, 01:24:31): K-means'te veride kaç küme olduğunu bilmiyorsak k'yi nasıl seçeriz?

Çözüm: $k = 2, 3, 4, \ldots$ denenir, her sonuç için merkeze ortalama uzaklık gibi bir kalite ölçüsü hesaplanır. k bir artınca düşüşün belirgin biçimde yavaşladığı değer seçilir.

Soru 5 (derste sözlü, 01:37:02): Veriyi tek geçişte kümeleyebilir miyiz?

Çözüm: Derste cevap BFR algoritmasıyla verildi: Veri belleğe sığan parçalar hâlinde bir kez okunur. Her nokta Mahalanobis uzaklığına göre ya bir kümeye eklenip atılır (yalnız N, SUM, SUMSQ güncellenir) ya da bellekte tutulur.

Soru 6 (derste sözlü, 01:56:31): d boyutlu veride, discard set'teki her küme için bellekte kaç sayı tutulur?

Çözüm: $2d + 1$. SUM vektörü d, SUMSQ vektörü d sayıdır, N ise tek bir sayıdır.

Soru 7 (derste sözlü, 02:12:00): Bir nokta, d boyutlu bir kümenin merkezinden her boyutta tam bir standart sapma uzaktaysa Mahalanobis uzaklığı nedir? 2σ ve 3σ için nedir?

Çözüm: Her boyutta $y_i = 1$ olur ve uzaklık $\sqrt{d}$'dir. 2σ için $2\sqrt{d}$, 3σ için $3\sqrt{d}$'dir. Bu değerler üyelik eşiği olarak kullanılabilir.

Dersin Toparlaması

Bu derste kümeleme, aralarında bir uzaklık kavramı tanımlı noktaları birbirine yakın gruplara ayırma problemi olarak ele alındı. Uzaklık ölçüsü olarak Jaccard, Öklid, kosinüs ve BFR'de kullanılan Mahalanobis uzaklığı görüldü. Kümelemenin iki ana ailesi işlendi. Hiyerarşik (birleştirici) kümeleme k istemez ve iyi sonuç verir, ama $O(n^3)$, dikkatli uygulamada $O(n^2 \log n)$ karmaşıklığı ve bütün verinin bellekte olması şartı yüzünden yalnız küçük veride kullanılabilir. K-means Öklid uzayında çalışır ve k ister; k bilinmiyorsa merkeze ortalama uzaklık grafiğiyle seçilir, başlangıç noktaları örnekleme ya da dağınık noktalar yöntemiyle bulunur.

Belleğe sığmayan veri için iki algoritma görüldü. BFR, K-means'in büyük veri sürümüdür. Her boyutta normal dağılım varsayar ve kümeleri yalnız $2d+1$ sayılık özet istatistiklerle (N, SUM, SUMSQ) bellekte tutar. Noktaları Mahalanobis uzaklığı ve $\sqrt{d}$ katı eşiklerle kümelere atar. CURE ise normal dağılım varsayımını kaldırır. Bir örnek üzerinde hiyerarşik kümeleme yapar, her kümeden dağınık temsilci noktalar seçip merkeze doğru kaydırır ve bütün veriyi en yakın temsilciye atar. Böylece her biçimdeki kümeleri bulabilir. Ders slaytları (Clustering, Hierarchical Clustering, K-Means, BFR ve CURE) dersin web sayfasında paylaşıldı.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.